Open in Colab

Lab 8.1 Machine Learning 1 – Artificial Neural Network, Classification

Learning goals

Students will be able to:

1. Create code to plot time and frequency domain data for normal and abnormal data
2. Implement an artificial neural network to classify different datasets

Introduction

In this lab, we will create a machine learning (ML) model based on the accelerometer (ADXL345) signals to predict the running conditions of the axial flow fan (AFF) which we used in Lab 2. An autoencoder we practiced in Prelab8 will be employed to determine whether AFF is in a normal or abnormal condition. Lab8 is broken down into two main sections: **1) Data collection**, and **2) Training ML model**. In the data collection section, we will collect accelerometer data when the machine is in normal and abnormal conditions using Raspberry Pi. The abnormal condition will be set of increasing eccentric force by adding a mass on a blade of the AFF as Lab2. In the training ML model section, we will utilize the given scripts in Prelab8 to train an autoencoder. In addition, we will perform feature engineering by doing signal processing to see the effects of input feature on the performance of the ML models. And the, we will save the selected and trained model to local disk to use it in the next lab. The schematic of Lab9 is illustrated in Figure 1.

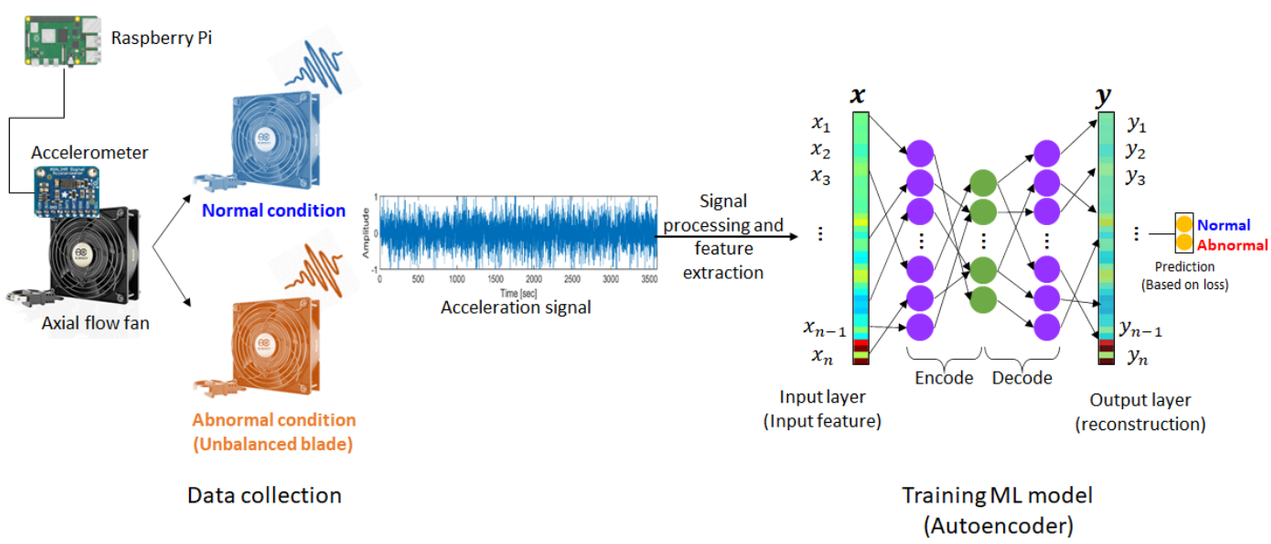


Figure 1 Lab8: Schematic of anomaly detection model training for axial flow fan

Data Collection for Machine Learning

Part 1: Data collection practice

First, make a wire connection between ADXL345 sensor and Raspberry Pi. If you are having trouble with the connection, please look at the instructions in Lab2 manual. After you make a connection,

you are ready to collect data. Before attaching the sensor to the target placement, let's try to get data and understand the output data format.

The sample Python script (Lab8_data_collector.py) below to collect ADXL345 accelerometer data is on Github repo. In the script, what you need to pay attention to is 'condition_identifier' and 'duration' variables in the middle of the script.

Python - Python3 (Lab8_data_collector.py)

```
import time
import board
import busio
import adafruit_adxl34x
from micropython import const
import csv
import datetime

i2c = busio.I2C(board.SCL, board.SDA) # i2c variable defines I2C interfaces and GPIO pins us

acc = adafruit_adxl34x.ADXL345(i2c) # acc object is instantiation using i2c of Adafruit ADXL

acc.data_rate = const(0b1111) # change sampling rate as 3200 Hz

# ratedict=output rate dictionary
# See Table5 of Lab2 manual key=rate code (decimal), value=output data rate (Hz)
ratedict = {15:3200,14:1600,13:800,12:400,11:200,10:100,9:50,8:25,7:12.5,6:6.25,5:3.13,4:1.5

print("Output data rate is {} Hz".format(ratedict[acc.data_rate])) # printing out data rate

def getData(sensor:object, N:int): # sensor: ADXL sensor object, N: The number of sample in
    t1 = time.time()
    data_x = [] # initialize data_x to contain x-axis acceleration
    data_y = [] # initialize data_y to contain y-axis acceleration
    data_z = [] # initialize data_z to contain z-axis acceleration
    for i in range(N):
        x_acc, y_acc, z_acc = sensor.acceleration
        data_x.append(str(x_acc))
        data_y.append(str(y_acc))
        data_z.append(str(z_acc))
    x_data = ' '.join(data_x)
    y_data = ' '.join(data_y)
    z_data = ' '.join(data_z)
    return x_data, y_data, z_data # each data returns space delimited string each element is

condition_identifier = "Test" # condition identifier
duration = 10 # data collection duration in second unit

filename = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")+"_"+condition_identifier+"_lab8
header = ["Condition", "Xacc array [m/s2]", "Yacc array [m/s2]", "Zacc array [m/s2]"]
start = time.time()

print("== Data collection for {} measurements started. ==".format(duration))

with open(filename, 'w') as f: # Make and open file object
    write = csv.writer(f) # write object for the created file
    write.writerow(header) # write the first row (header)
    for j in range(duration): # for measurement durations
        x, y, z = getData(acc, 1000) # get x-, y-, z-axis acceleration array of 1000 data po
```

```
        print('======= {}th of {} collection ========'.format(j+1, duration)) # Print out the
        write.writerow([condition_identifier, x, y, z])
f.close()

print("== Data saving is done. == takes", time.time() - start)
```

Let's try to collect data as a practice. Please set 'condition_identifier' as "test" and 'duration' as 10. If you run the script, your Raspberry Pi will collect accelerations for 10 seconds. According to Raspberry Pi specifications, the total time may be longer than the 'duration' you set. The output filename must be "YYYYMMDD_HHmmSS_Test_lab8_data.csv". The date-time in the first part of the filename is the date-time when the script starts to be run. The "Test" in this case is the 'conditon_identifier'. If you open the saved CSV file, you will see the collected data as Table 1. The first column (Condition column) indicates 'condition_identifier'. The second, third, and last columns indicate the measured accelerations for 1 second (1000 data points) for each axis, respectively. Because the sampling frequency is 1000Hz, each row means data for 1 second. Each array is space delimited float array. To practice data loading and transformation, perform Task 1.

Table 1 Example of collected data

Condition	Xacc array [m/s2]	Yacc array [m/s2]	Zacc array [m/s2]
		...	
Test	x_1 x_2 ... x_{999} x_{1000}	y_1 y_2 ... y_{999} y_{1000}	z_1 z_2 ... z_{999} z_{1000}
Test	x_1 x_2 ... x_{999} x_{1000}	y_1 y_2 ... y_{999} y_{1000}	z_1 z_2 ... z_{999} z_{1000}
Test	x_1 x_2 ... x_{999} x_{1000}	y_1 y_2 ... y_{999} y_{1000}	z_1 z_2 ... z_{999} z_{1000}
		...	
Test	x_1 x_2 ... x_{999} x_{1000}	y_1 y_2 ... y_{999} y_{1000}	z_1 z_2 ... z_{999} z_{1000}
Test	x_1 x_2 ... x_{999} x_{1000}	y_1 y_2 ... y_{999} y_{1000}	z_1 z_2 ... z_{999} z_{1000}
Test	x_1 x_2 ... x_{999} x_{1000}	y_1 y_2 ... y_{999} y_{1000}	z_1 z_2 ... z_{999} z_{1000}
		...	

Task 1.1

After running ‘Lab8_data_collector.py’ with variables, ‘condition_identifier’=”Test” and ‘duration’=10, plot each axis data in both time-domain and frequency-domain as Figure 2. An example Python sciript for plotting data is given (Lab8_plot_example.py). You can use the given Python script to plot data. However, it is strongly recommended to understand each line of the script.

- Capture the plot and attach it to the report below.
    - Load the CSV file you collected in Python script.
    - Add ‘condition_identifier’ and your name at the end of the title of each plot (e.g., ‘Time domain, Test, John Doe’).
    - Your plots must include each axis label and units.
    - Recall Lab2 and Prelab8. You have already done this work before.
    - Your data have 10 rows except the header row because you collected data for 10 seconds. Please select one of the data rows randomly.
    - You can use either Raspberry Pi or laptop to plot the data. If you want, perform plotting the given code in the code block below this Colab Notebook.

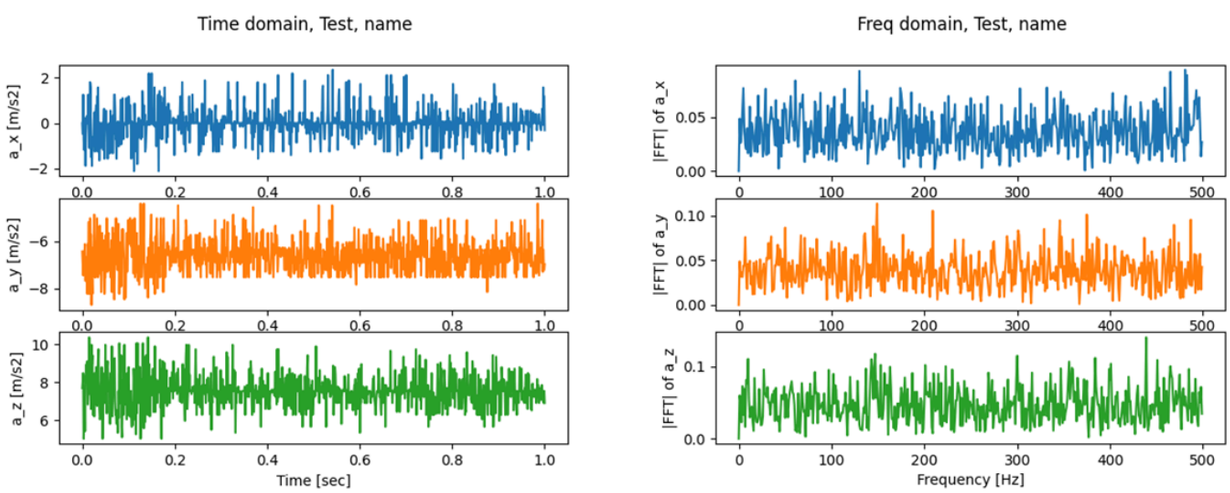
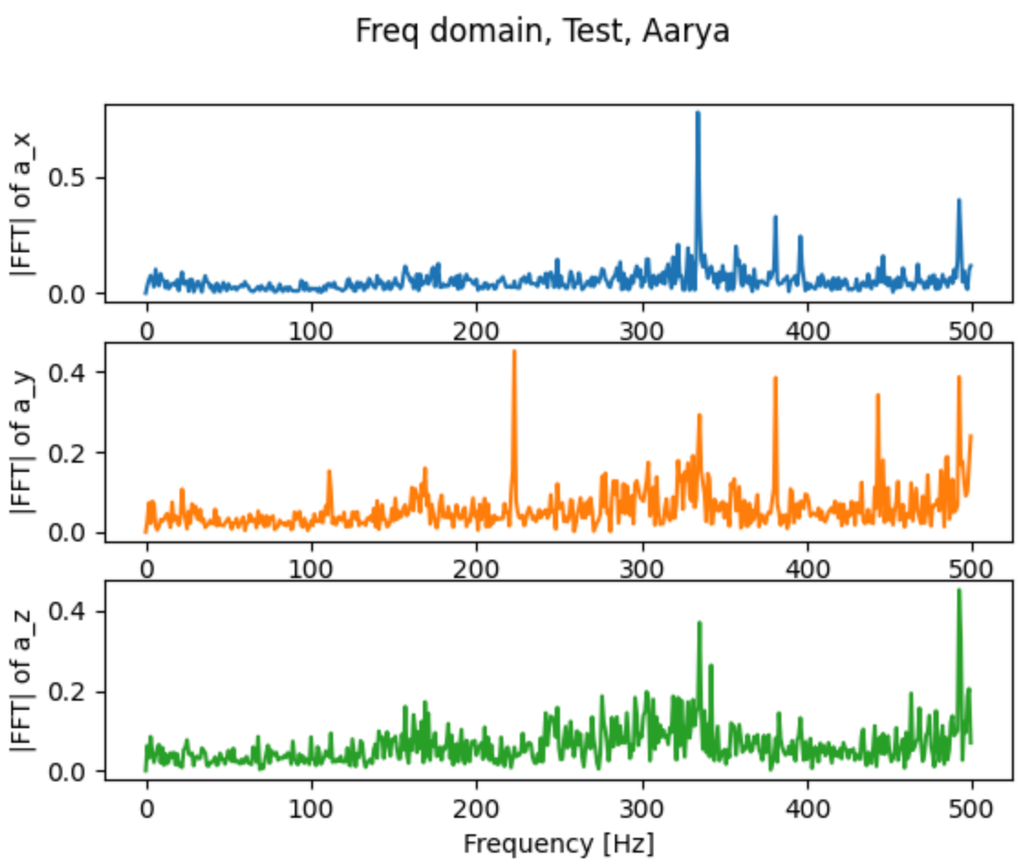


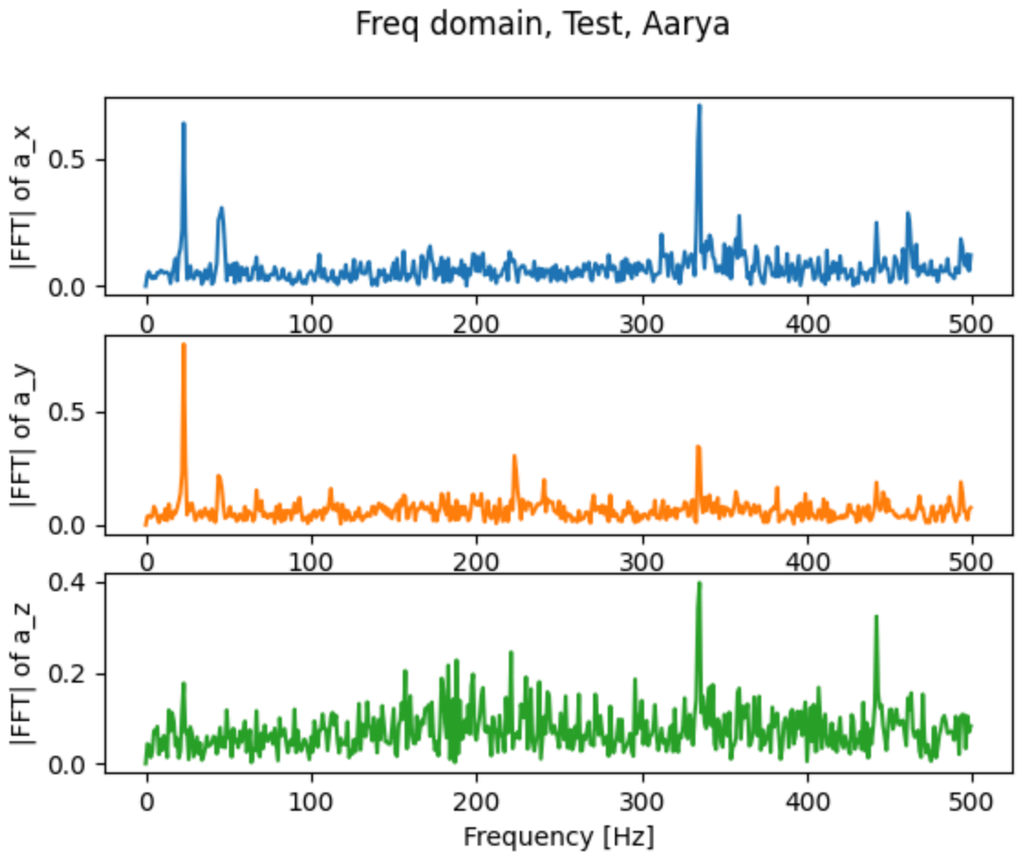
Figure 2 Data plot: Time domain (left) and Frequency domain (right)

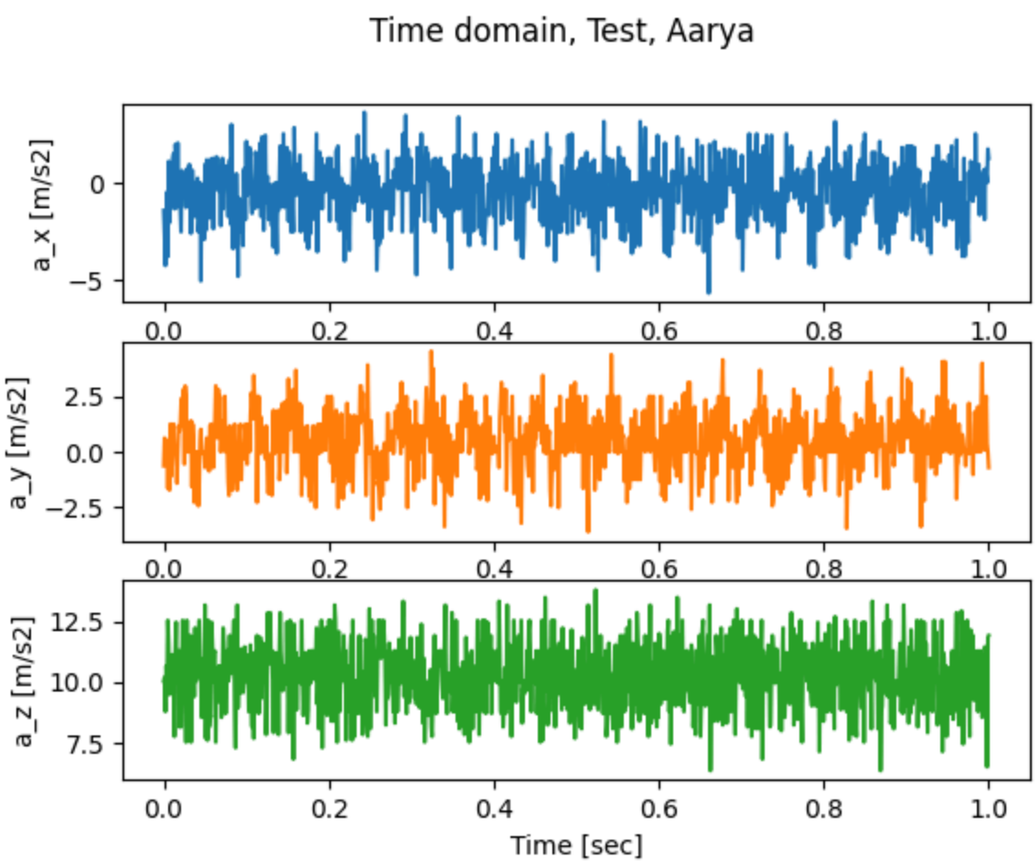
normal :

Time domain, Test, Aarya
a_x [m/s2]
a_y [m/s2]
a_z [m/s2]
Time [sec]


Freq domain, Test, Aarya
|FFT| of a_x
|FFT| of a_y
|FFT| of a_z
Frequency [Hz]

Abnoraml :





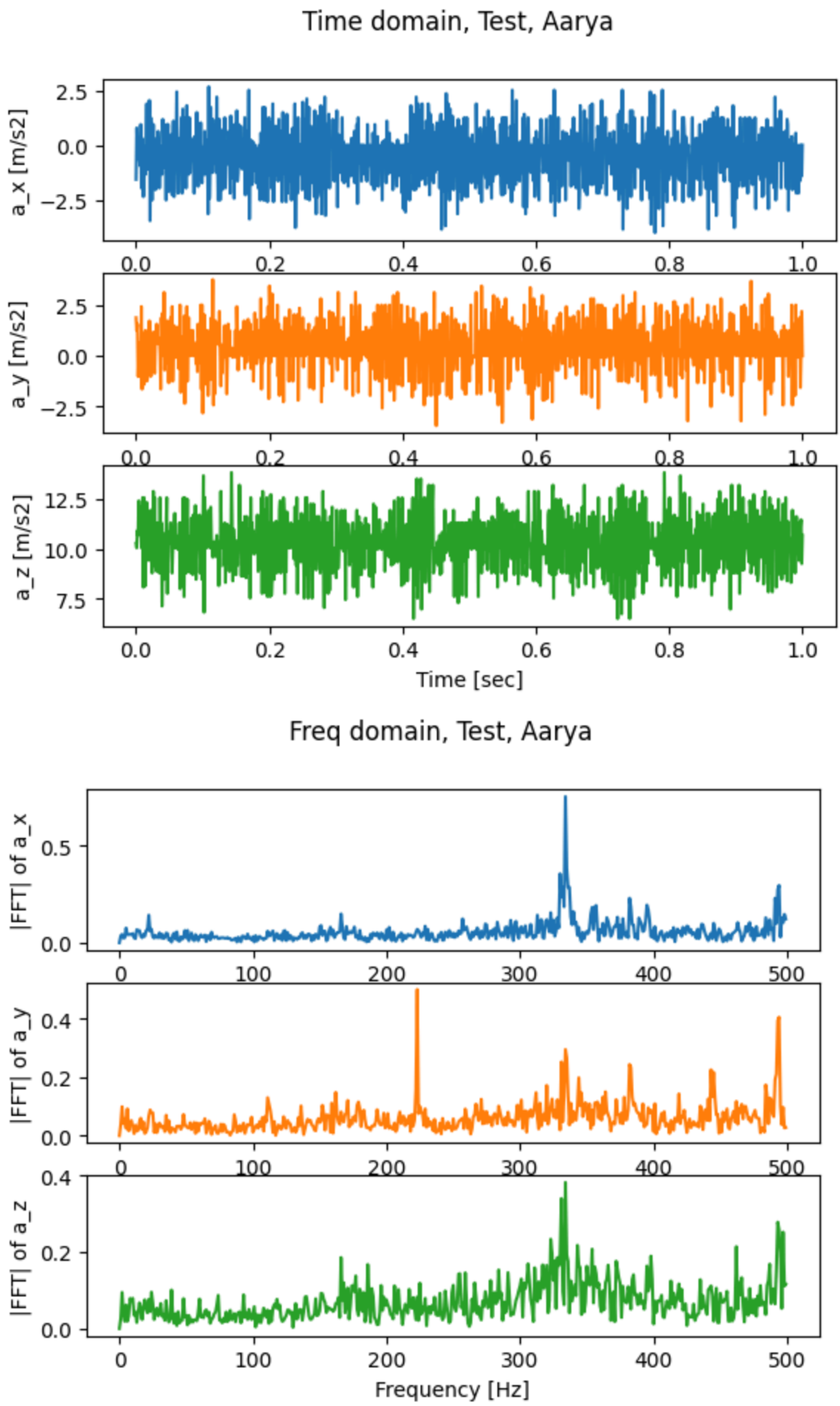

```
from google.colab import files
uploaded = files.upload()
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving abnormal_Test_lab8_data.csv to abnormal_Test_lab8_data (1).csv
Saving normal_Test_lab8_data.csv to normal_Test_lab8_data (1).csv
Saving speed2_Abnormal_lab8_data.csv to speed2_Abnormal_lab8_data.csv
Saving speed2_normal_lab8_data.csv to speed2_normal_lab8_data.csv
Saving speedH_Abnormal_lab8_data.csv to speedH_Abnormal_lab8_data.csv
Saving speedH_normal_lab8_data.csv to speedH_normal_lab8_data.csv
```

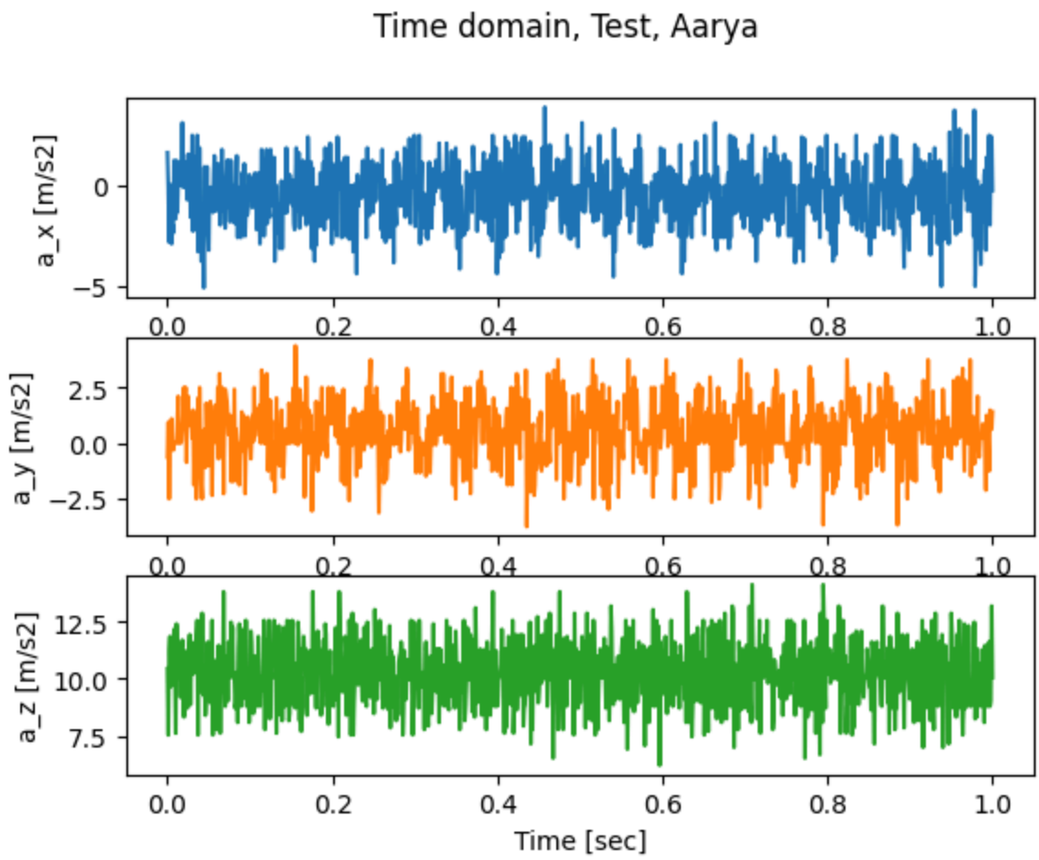
```
import matplotlib.pyplot as plt
import csv
import numpy as np
import random
from scipy.fft import fft, fftfreq

# You should change filename.
# If the data is in the different location of this Python script,
# you have to specify the directory information as well.
filename = "normal_Test_lab8_data.csv" # input your filename
data = [] # initialize data
N = 1000 # number of sample
T = 1/N # sampling period
R = random.randint(0, 9) # randomly pick a integer number between 0 and 10
t = np.linspace(0, 1, N) # create time array for plotting

# Reading a CSV file and convert rows as data
with open(filename, 'r') as f:
    rows = csv.reader(f, delimiter = ',')
    header = next(rows)
    for row in rows:
        data.append(row)

data = np.array(data) #converting data as numpy array


def decompose(data): # decompose data from string (with space delimited) to a float array
    return np.fromstring(data, dtype=float, sep=" ")


a_x = decompose(data[R][1]) # X-axis acceleration array
a_y = decompose(data[R][2]) # Y-axis acceleration array
a_z = decompose(data[R][3]) # Z-axis acceleration array

condition = data[R][0] # get the condition string

your_name = "Aarya" # input your name
Time_title = "Time domain, "+condition+", "+your_name
Freq_title = "Freq domain, "+condition+", "+your_name

fig1, ax1 = plt.subplots(3) # plotting time domain for 3-axis
fig1.suptitle(Time_title)
ax1[0].plot(t,a_x,'tab:blue')
ax1[0].set_ylabel('a_x [m/s2]')
ax1[1].plot(t,a_y, 'tab:orange')
ax1[1].set_ylabel('a_y [m/s2]')
ax1[2].plot(t,a_z, 'tab:green')
ax1[2].set_ylabel('a_z [m/s2]')
ax1[2].set_xlabel('Time [sec]')


def get_fft(x, T, N): # fft converting
    f = fftfreq(N, T)[:N//2]
    y_mag = 2/N * np.abs(fft(x)[:N//2])
    y_mag[0] = 0
    return f, y_mag

f_x, y_x = get_fft(a_x, T, N)
f_y, y_y = get_fft(a_y, T, N)
f_z, y_z = get_fft(a_z, T, N)

fig2, ax2 = plt.subplots(3) #plotting freq domain for 3-axis
fig2.suptitle(Freq_title)
ax2[0].plot(f_x,y_x,'tab:blue')
ax2[0].set_ylabel('|FFT| of a_x')
ax2[1].plot(f_y,y_y,'tab:orange')
ax2[1].set_ylabel('|FFT| of a_y')
```

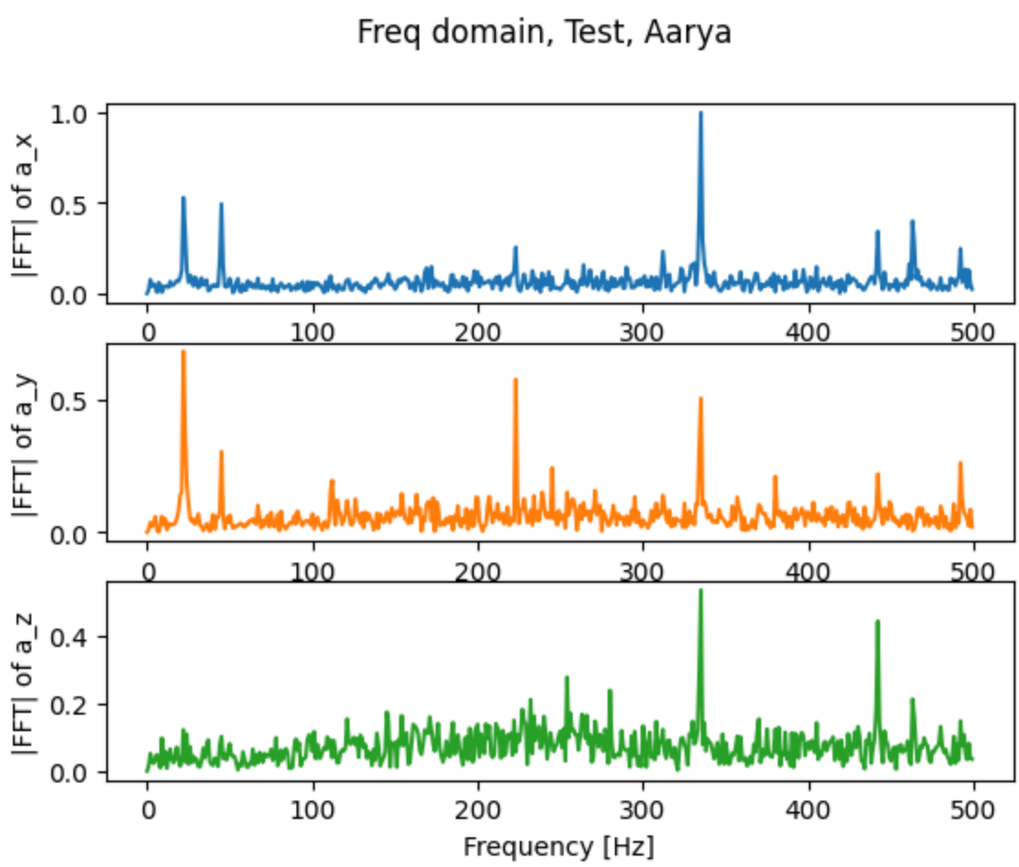
```
ax2[2].plot(f_z,y_z,'tab:green')
ax2[2].set_ylabel('|FFT| of a_z')
ax2[2].set_xlabel('Frequency [Hz]')

plt.show()
```




```
import matplotlib.pyplot as plt
import csv
import numpy as np
import random
from scipy.fft import fft, fftfreq

# You should change filename.
# If the data is in the different location of this Python script,
# you have to specify the directory information as well.
filename = "abnormal_Test_lab8_data.csv" # input your filename
```

```
data = [] # initialize data
N = 1000 # number of sample
T = 1/N # sampling period
R = random.randint(0, 9) # randomly pick a integer number between 0 and 10
t = np.linspace(0, 1, N) # create time array for plotting

# Reading a CSV file and convert rows as data
with open(filename, 'r') as f:
    rows = csv.reader(f, delimiter = ',')
    header = next(rows)
    for row in rows:
        data.append(row)

data = np.array(data) #converting data as numpy array


def decompose(data): # decompose data from string (with space delimited) to a float array
    return np.fromstring(data, dtype=float, sep=" ")


a_x = decompose(data[R][1]) # X-axis acceleration array
a_y = decompose(data[R][2]) # Y-axis acceleration array
a_z = decompose(data[R][3]) # Z-axis acceleration array

condition = data[R][0] # get the condition string

your_name = "Aarya" # input your name
Time_title = "Time domain, "+condition+", "+your_name
Freq_title = "Freq domain, "+condition+", "+your_name

fig1, ax1 = plt.subplots(3) # plotting time domain for 3-axis
fig1.suptitle(Time_title)
ax1[0].plot(t,a_x,'tab:blue')
ax1[0].set_ylabel('a_x [m/s2]')
ax1[1].plot(t,a_y, 'tab:orange')
ax1[1].set_ylabel('a_y [m/s2]')
ax1[2].plot(t,a_z, 'tab:green')
ax1[2].set_ylabel('a_z [m/s2]')
ax1[2].set_xlabel('Time [sec]')


def get_fft(x, T, N): # fft converting
    f = fftfreq(N, T)[:N//2]
    y_mag = 2/N * np.abs(fft(x)[:N//2])
    y_mag[0] = 0
    return f, y_mag

f_x, y_x = get_fft(a_x, T, N)
f_y, y_y = get_fft(a_y, T, N)
f_z, y_z = get_fft(a_z, T, N)

fig2, ax2 = plt.subplots(3) #plotting freq domain for 3-axis
fig2.suptitle(Freq_title)
ax2[0].plot(f_x,y_x,'tab:blue')
ax2[0].set_ylabel('|FFT| of a_x')
ax2[1].plot(f_y,y_y,'tab:orange')
ax2[1].set_ylabel('|FFT| of a_y')
ax2[2].plot(f_z,y_z,'tab:green')
ax2[2].set_ylabel('|FFT| of a_z')
ax2[2].set_xlabel('Frequency [Hz]')

plt.show()
```





Part 2: Data collection in normal condition

Before we deploy the sensor to the AFF, let's check the hardware and the speed controller. The hardware configuration and the speed controller are shown in Figure 3. For safety reasons, the base part is fixed on the table using tapes. Do not remove the tapes because it maybe move when you run the AFF due to aerodynamic forces. To turn on the fan, rotate the knob of the speed controller clockwise. The relationship between knob pointer placements and the actual rotational speed of AFF is shown in Table 2. When you rotate the pointer of the knob to L (Speed 1 in Figure 3 (right)), for example, the AFF rotates around 1800 rpm. By adjusting the control knob, you can increase and decrease the rotating speed of the fan. Different from the vacuum pump case of Prelab8, it is obvious that changing rotational speed makes an ML model development hard. On the

other hand, it may be more interesting. For instance, if you attach the unbalanced mass to a blade of the fan at the lowest rotational speed, does the amplitude of vibration be bigger than the maximum rotational speed without any attached mass? We don't know the answer exactly but probably no. Turn the fan on and adjust the knob to see if the rotation speed changes well.

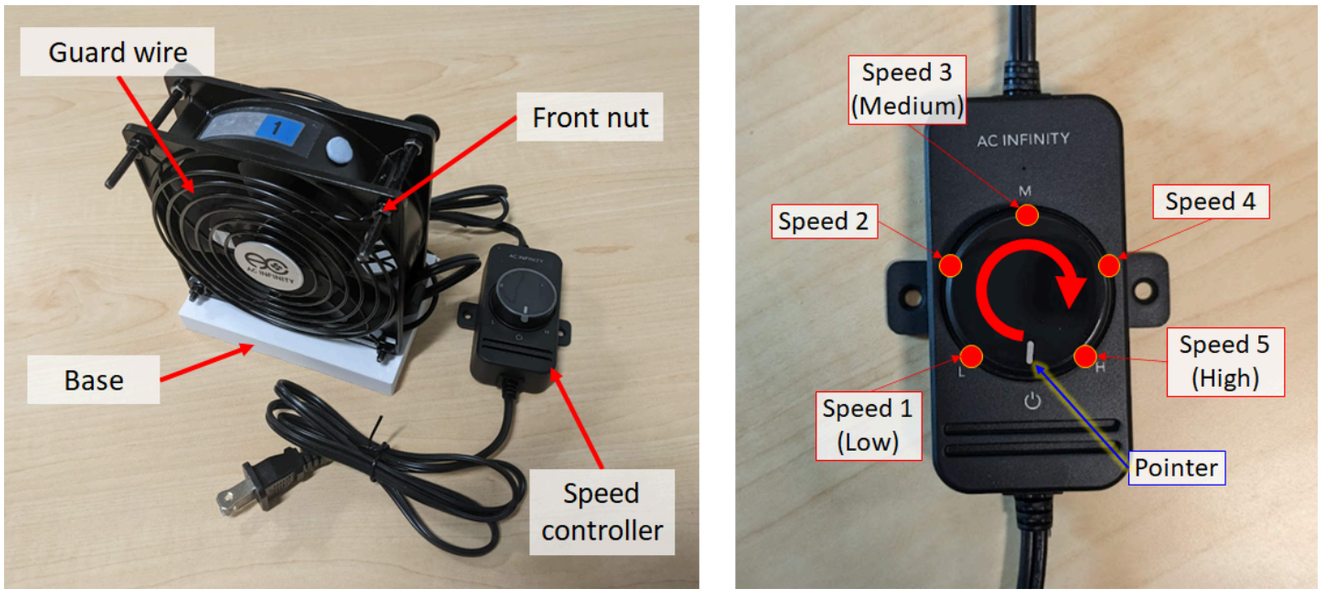


Figure 3 Configuration of axial flow fan (left) and speed controller (right)

Table 2 Controller indicator vs. rotational speed of AFF

Speed indiator	Rotational Speed
1 (L, Low)	1800 rpm
2	2150 rpm
3 (M, Middle)	2500 rpm
4	2750 rpm
5 (H, High)	3000 rpm

Deploy the accelerometer (ADXL345) on the top of the fan as Figure 4. **You have to remember the sensor placement and the axis configuration for the next lab again.** If you are not sure how to set up the accelerometer to the AFF, please look at Lab2.2 manual. To collect acceleration data when the machine is in normal conditions for 5 minutes, perform Task 1.2. While you collect the data, try to change the rotational speed. For example, while collecting, set Speed 1 (1800 rpm) for the first 1 minute, and then Speed 2 (2150 rpm) between 1 and 2 minutes and so forth. The example of the experimental table is shown in Table 3. You see the progress of your data collector in the Shell of Thonny or Terminal window of Raspberry Pi as Figure 5. For this, please perform data collection at once with other members sharing the table and the AFF.

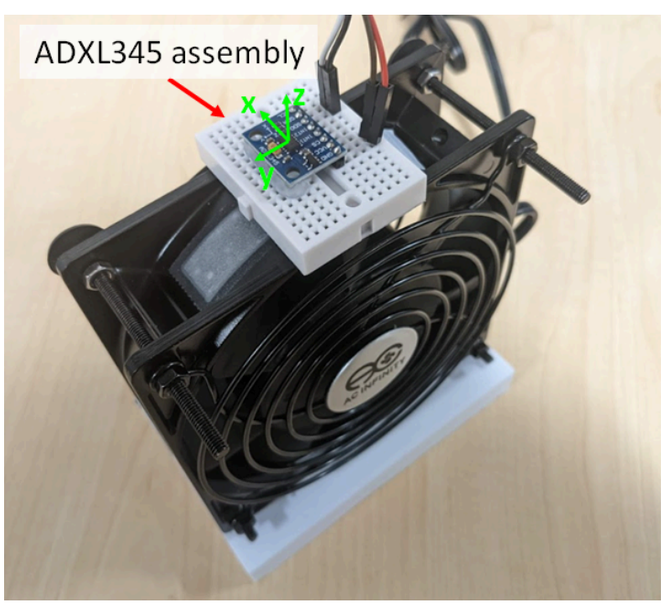


Figure 4 Sensor placement on top of the AFF

Table 3 Experimental table for data collection of AFF

Measurement	Time (Approximately)	Speed
1 - 60	0 - 60 seconds	1 (L, 1800 rpm)
61 - 120	61 - 120 seconds	2 (2150 rpm)
121 - 180	121 - 180 seconds	3 (M, 2500 rpm)
181 - 240	181 - 240 seconds	4 (2750 rpm)
241 - 300	241 - 300 seconds	5 (3000 rpm)

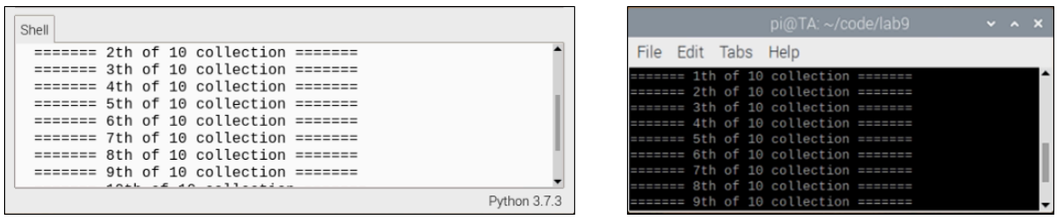


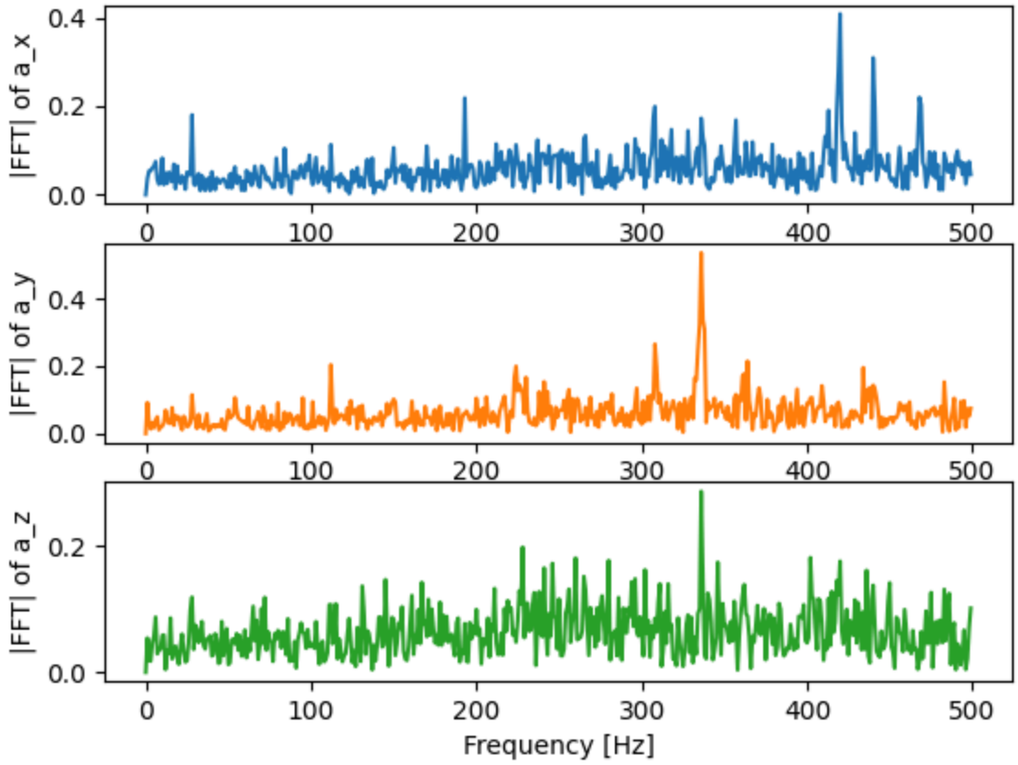
Figure 5 Data collection progress check: Thonny (left) and Terminal (right)

Task 1.2

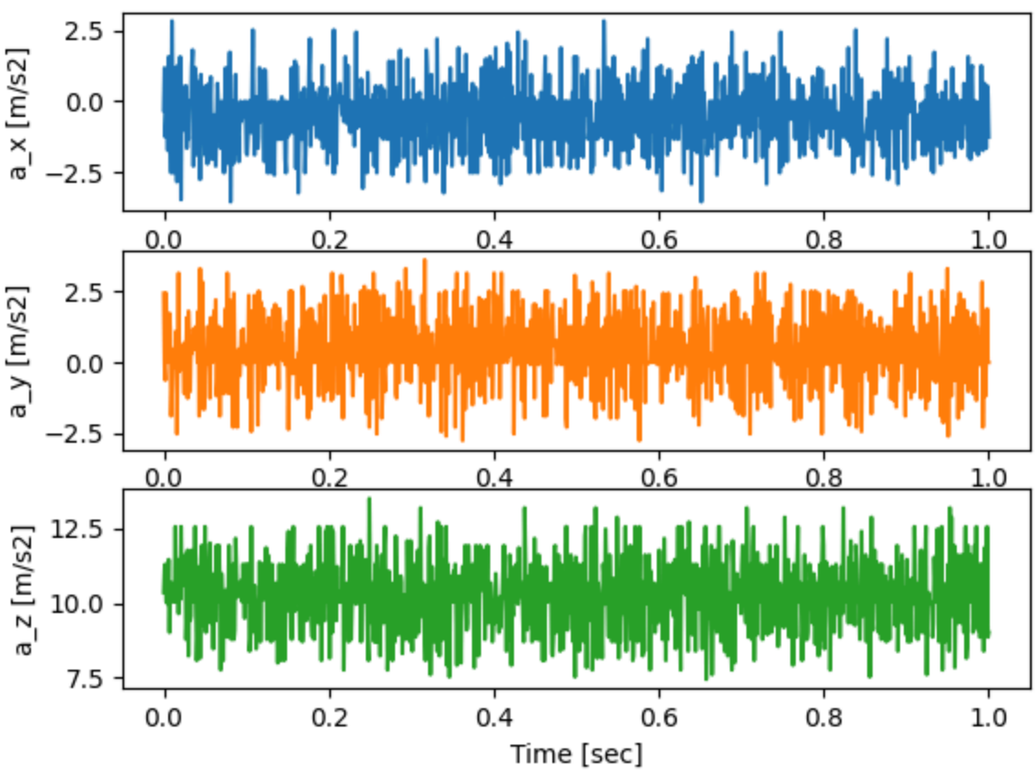
Run 'Lab8_data_collector.py' while you run the AFF with variables, 'condition_identifier'="Normal" and 'duration'=300, plot each axis data in both time-domain and frequency-domain as Figure 2.

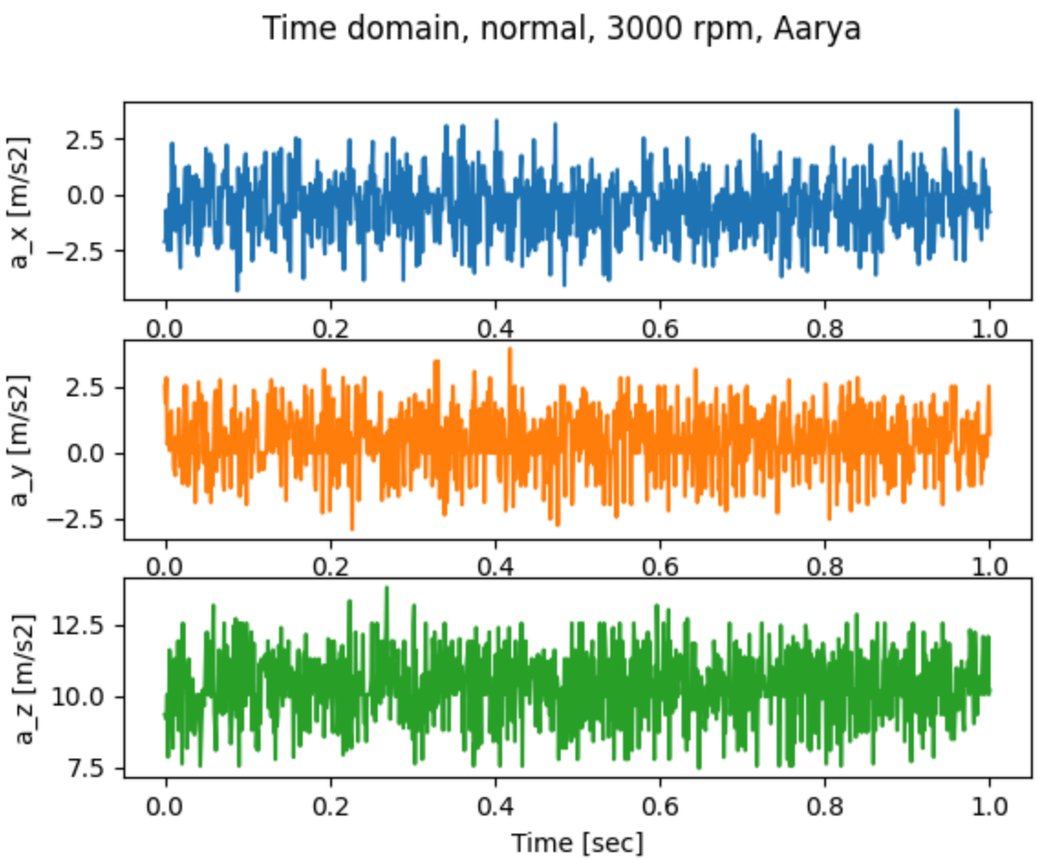
- Capture the plots according to the rotational speed and attach these to the report below.
  - Load the CSV file in Python script.
  - Add 'condition_identifier', rotational speed, and your name at the end of the title of each plot (e.g., 'Time domain, Normal, 1800 rpm, John Doe').
  - Your plots must include each axis label and units.
  - **Plot two different rotational speeds and compare them** For example, one is 1800 rpm and another is 2500 rpm.
- Your data have 300 rows except the header row because you collected data for 300 seconds. Note that the actual collection time is probably loger than 300 seconds because of data saving time. Please select one of the data rows randomly within one-speed range. For example, if you plot Speed 2, you need to select a data row between 61 and 120.

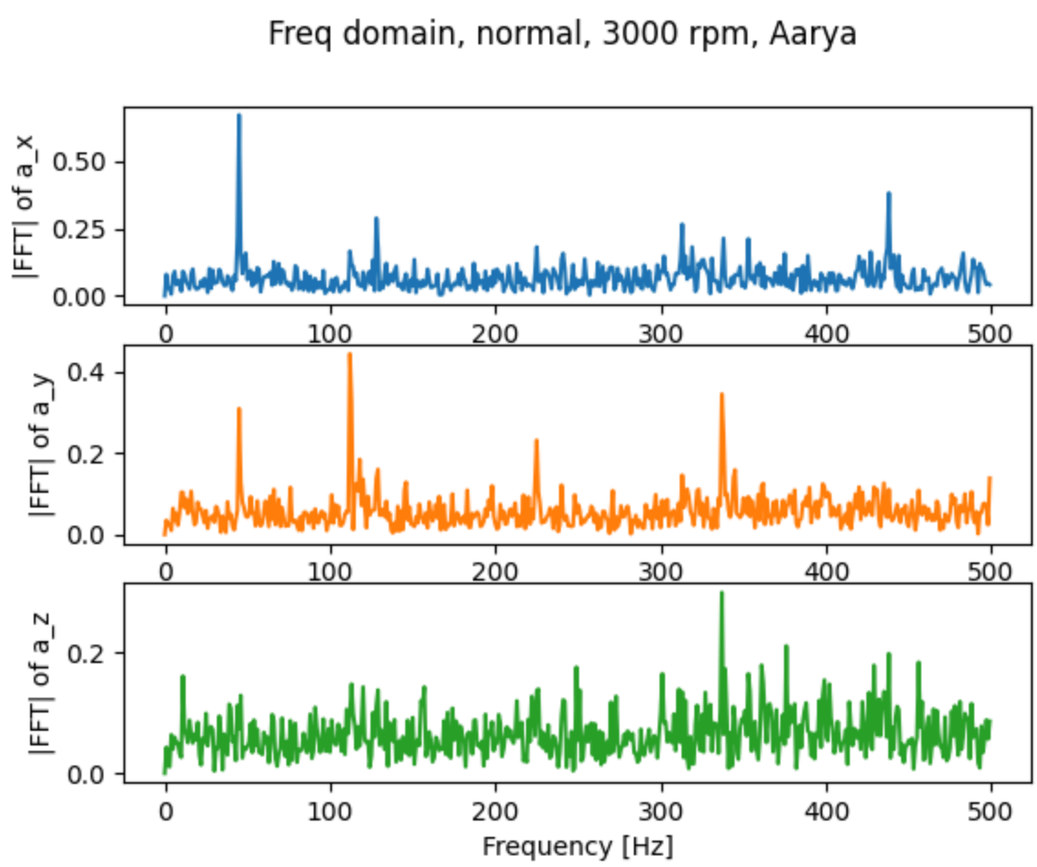
Freq domain, normal, 2150 rpm, Aarya



Time domain, normal, 2150 rpm, Aarya





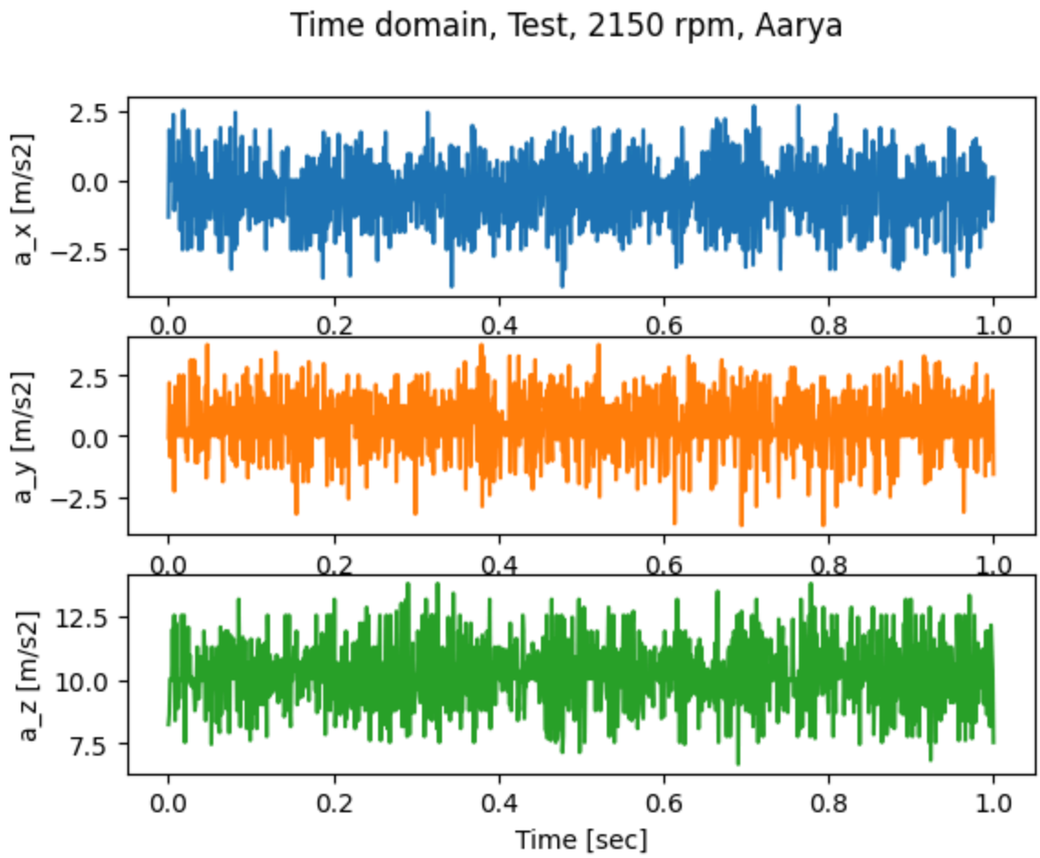


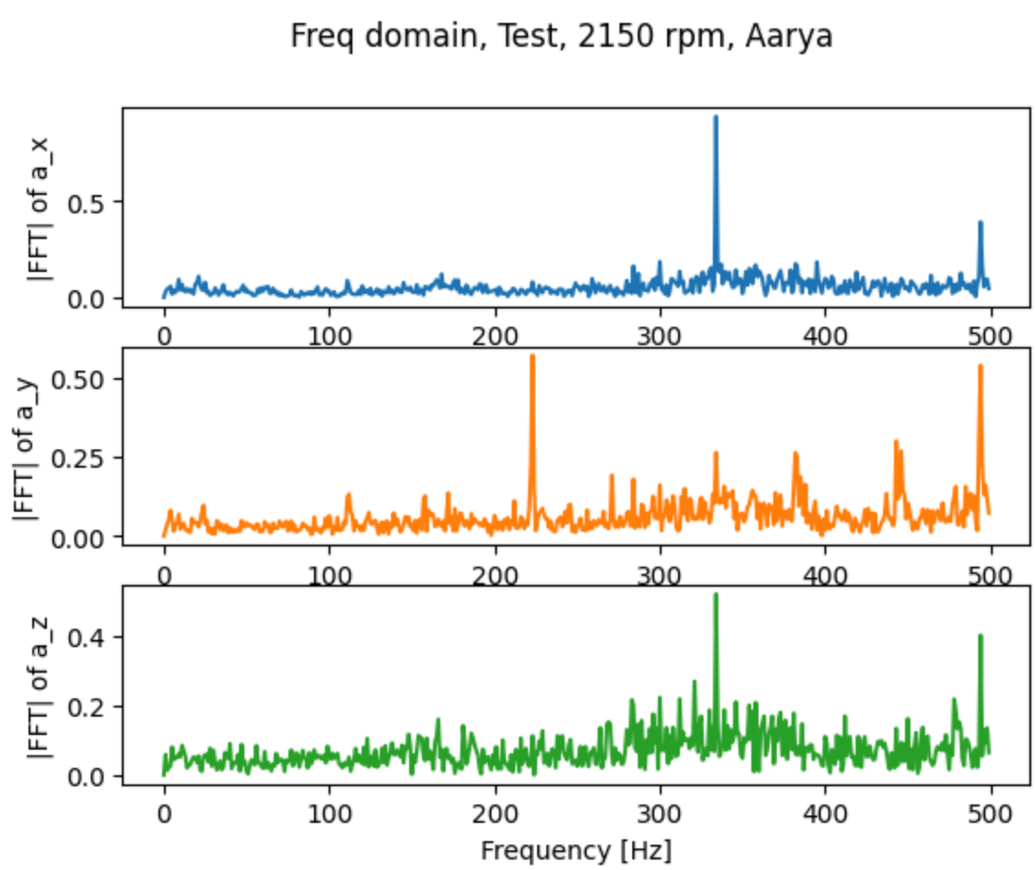

```
# Perform Task 1.2 for plotting "Test" data here
# If you want to use this Colab Notebook, you have to load your file on
# 1) the Colab virtual directory
# 2) mounting your Google Colab drive
#

import matplotlib.pyplot as plt

import csv

import numpy as np

import random

from scipy.fft import fft, fftfreq
```

```
# You should change filename.

# If the data is in the different location of this Python script,

# you have to specify the directory information as well.

filename = "normal_Test_lab8_data.csv" # input your filename

data = [] # initialize data

N = 1000 # number of sample

T = 1/N # sampling period

R = random.randint(0, 299) # randomly pick a integer number between 0 and 10

t = np.linspace(0, 1, N) # create time array for plotting


# Reading a CSV file and convert rows as data

with open(filename, 'r') as f:

    rows = csv.reader(f, delimiter = ',')

    header = next(rows)

    for row in rows:

        data.append(row)


data = np.array(data) #converting data as numpy array




def decompose(data): # decompose data from string (with space delimited) to a float array

    return np.fromstring(data, dtype=float, sep=" ")

a_x = decompose(data[R][1]) # X-axis acceleration array

a_y = decompose(data[R][2]) # Y-axis acceleration array

a_z = decompose(data[R][3]) # Z-axis acceleration array


condition = data[R][0] # get the condition string


your_name = "Aarya" # input your name

rpm = "2150 rpm"

Time_title = "Time domain, "+condition+", "+rpm +", "+your_name

Freq_title = "Freq domain, "+condition+", "+rpm +", "+your_name
```

```
fig1, ax1 = plt.subplots(3) # plotting time domain for 3-axis

fig1.suptitle(Time_title)

ax1[0].plot(t,a_x,'tab:blue')

ax1[0].set_ylabel('a_x [m/s2]')

ax1[1].plot(t,a_y, 'tab:orange')

ax1[1].set_ylabel('a_y [m/s2]')

ax1[2].plot(t,a_z, 'tab:green')

ax1[2].set_ylabel('a_z [m/s2]')

ax1[2].set_xlabel('Time [sec]')




def get_fft(x, T, N): # fft converting

    f = fftfreq(N, T)[:N//2]

    y_mag = 2/N * np.abs(fft(x)[:N//2])

    y_mag[0] = 0

    return f, y_mag


f_x, y_x = get_fft(a_x, T, N)

f_y, y_y = get_fft(a_y, T, N)

f_z, y_z = get_fft(a_z, T, N)


fig2, ax2 = plt.subplots(3) #plotting freq domain for 3-axis

fig2.suptitle(Freq_title)

ax2[0].plot(f_x,y_x,'tab:blue')

ax2[0].set_ylabel('|FFT| of a_x')

ax2[1].plot(f_y,y_y,'tab:orange')

ax2[1].set_ylabel('|FFT| of a_y')

ax2[2].plot(f_z,y_z,'tab:green')

ax2[2].set_ylabel('|FFT| of a_z')

ax2[2].set_xlabel('Frequency [Hz]')


plt.show()
```

```
#
```


Time domain, Test, 2150 rpm, Aarya
a_x [m/s2]
a_y [m/s2]
a_z [m/s2]
Time [sec]


Freq domain, Test, 2150 rpm, Aarya
|FFT| of a_x
|FFT| of a_y
|FFT| of a_z
Frequency [Hz]

Part 3: Data collection in abnormal condition

Next step is to collect abnormal condition data by adding mass on the blade for the ML model. There will be an adhesive putty on top of the AFF. Using the putty, make the fan unbalanced as Figure 6. If you are not sure how to set up the abnormal condition by adding mass, please review Lab2.2 manual. Please safely disassemble and assemble the AFF. If you are unsure about your setup, please ask TA. After setup for the abnormal condition of the AFF, perform TASK 3.

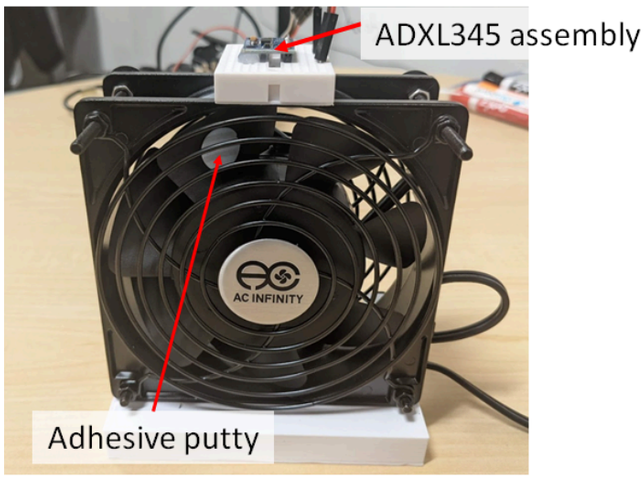
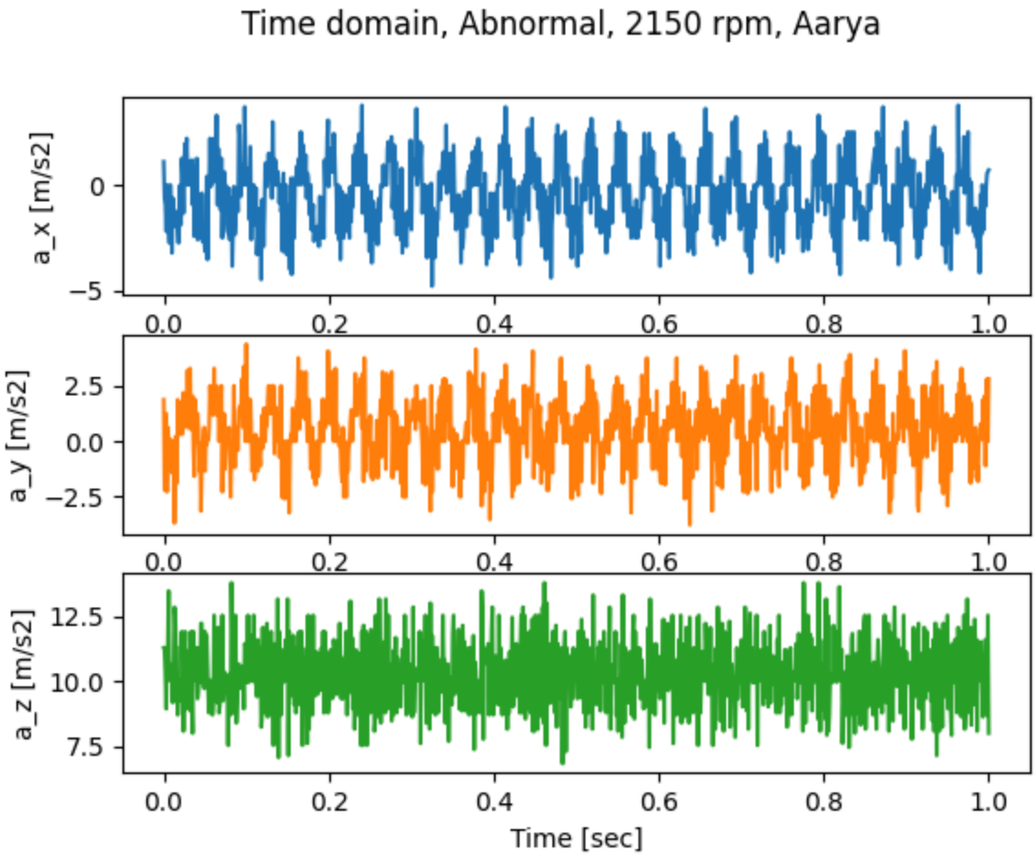


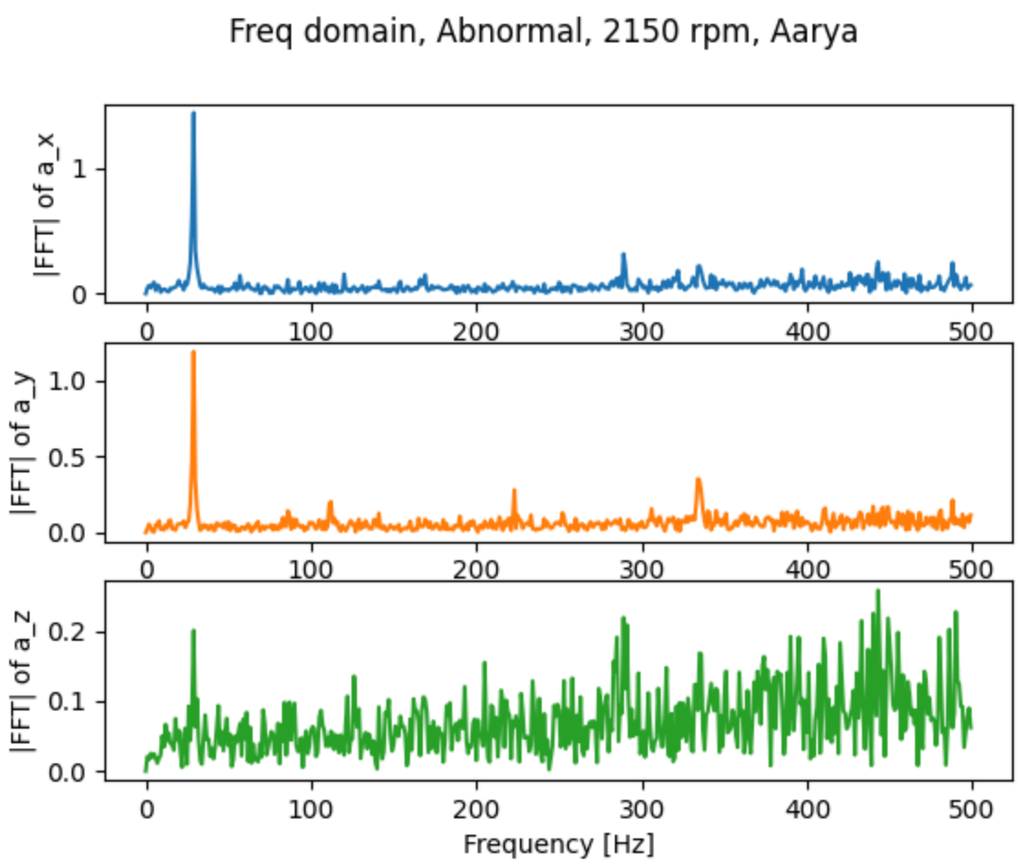
Figure 6 Unbalanced fan blade for abnormal condition of AFF

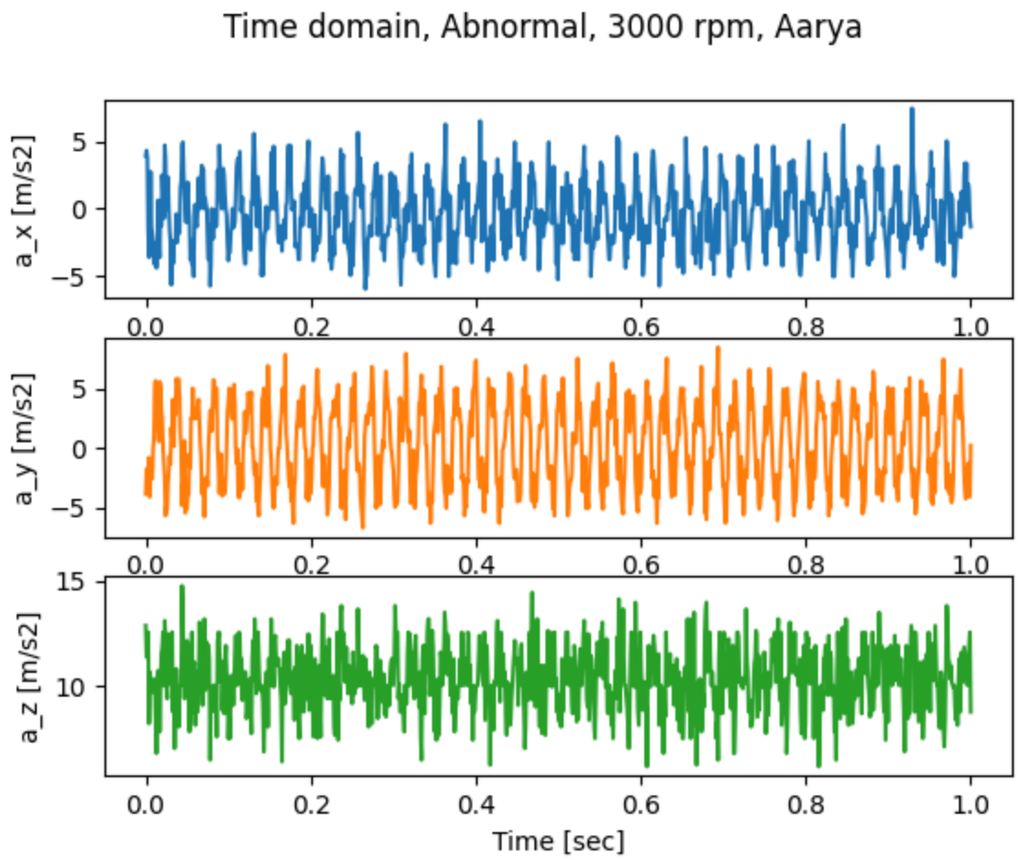
Task 1.3 (Repeat Task 1.2 in *abnormal* condition)

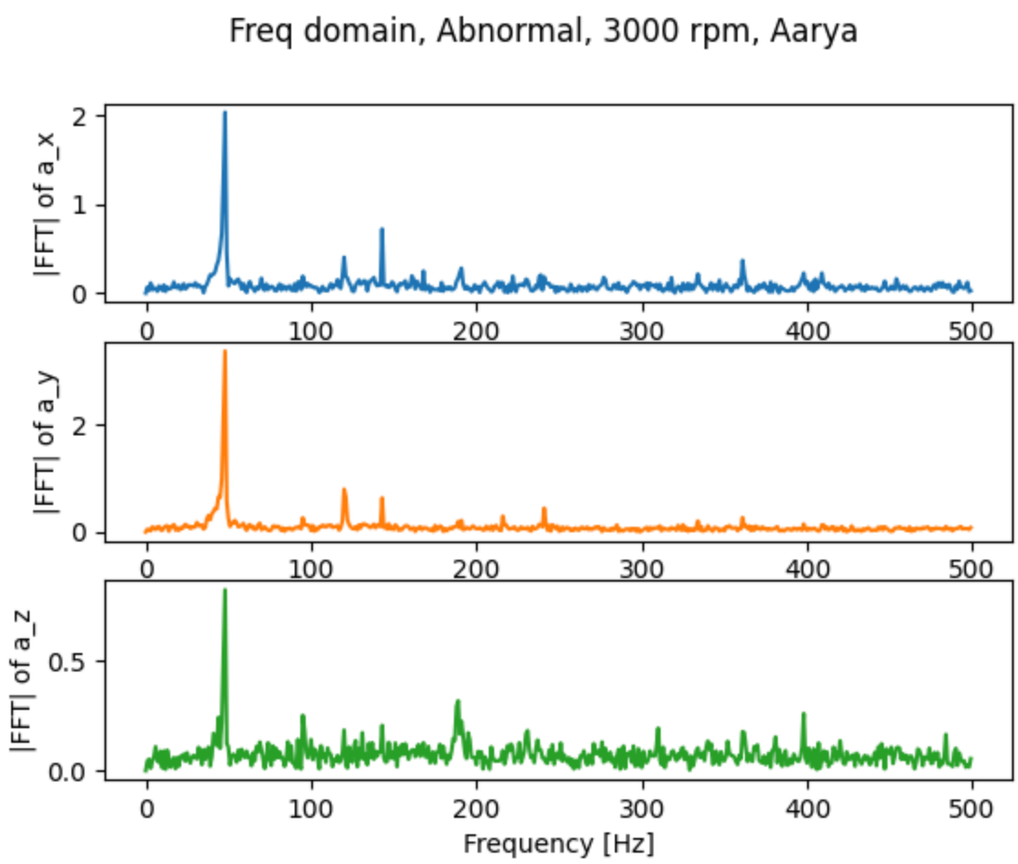
Run 'Lab8_data_collector.py' while you run the AFF with variables, 'condition_identifier'="Abnormal" and 'duration'=300, plot each axis data in both time-domain and frequency-domain as Figure 2.

- Capture the plots according to the rotational speed and attach these to the report below.
  - Load the CSV file in Python script.
  - Add 'condition_identifier', rotational speed, and your name at the end of the title of each plot (e.g., 'Time domain, Abnormal, 1800 rpm, John Doe').
  - Your plots must include each axis label and units.
  - **Plot two different rotational speeds and compare them** For example, one is 1800 rpm and another is 2500 rpm.
- Your data have 300 rows except the header row because you collected data for 300 seconds. Note that the actual collection time is probably loger than 300 seconds because of data saving time. Please select one of the data rows randomly within one-speed range. For example, if you plot Speed 2, you need to select a data row between 61 and 120.


Time domain, Abnormal, 2150 rpm, Aarya
a_x [m/s2]
a_y [m/s2]
a_z [m/s2]
Time [sec]


Freq domain, Abnormal, 2150 rpm, Aarya
|FFT| of a_x
|FFT| of a_y
|FFT| of a_z
Frequency [Hz]





```
# Perform Task 1.3 for plotting "Test" data here
# If you want to use this Colab Notebook, you have to load your file on
# 1) the Colab virtual directory
# 2) mounting your Google Colab drive
#

import matplotlib.pyplot as plt

import csv

import numpy as np

import random

from scipy.fft import fft, fftfreq
```

```
# You should change filename.

# If the data is in the different location of this Python script,

# you have to specify the directory information as well.

filename = "abnormal_Test_lab8_data.csv" # input your filename

data = [] # initialize data

N = 1000 # number of sample

T = 1/N # sampling period

R = random.randint(0, 299) # randomly pick a integer number between 0 and 10

t = np.linspace(0, 1, N) # create time array for plotting



# Reading a CSV file and convert rows as data

with open(filename, 'r') as f:

    rows = csv.reader(f, delimiter = ',')

    header = next(rows)

    for row in rows:

        data.append(row)



data = np.array(data) #converting data as numpy array




def decompose(data): # decompose data from string (with space delimited) to a float arr

    return np.fromstring(data, dtype=float, sep=" ")

a_x = decompose(data[R][1]) # X-axis acceleration array

a_y = decompose(data[R][2]) # Y-axis acceleration array

a_z = decompose(data[R][3]) # Z-axis acceleration array


condition = data[R][0] # get the condition string


your_name = "Aarya" # input your name

rpm = "2150 rpm"

Time_title = "Time domain, "+condition+", "+rpm +", "+your_name

Freq_title = "Freq domain, "+condition+", "+rpm +", "+your_name
```